INFRAFLOW / CODEMANUS · ENGINEERING KNOWLEDGE TRANSFER

Authentication, End to End

How “Continue with Google / GitHub” (OAuth 2.0 + OpenID Connect) was built into this application, how every other authentication mechanism in the codebase works, and how to explain all of it in an interview.

1. The 60-second answer
2. Prerequisites — what you need before it works
3. Foundations: OAuth 2.0, OIDC, PKCE, JWT
4. Every auth mechanism in this app
5. The new flow — architecture & sequence
6. Code walk-through, step by step
7. Account linking & pre-account hijacking
8. Sessions, JWTs & revocation
9. Threat model: each defence and the attack it stops
10. Testing strategy
11. Production rollout checklist
12. Production-readiness review & known gaps
13. Interview questions & model answers
14. Cheat sheet & glossary
15. Token storage hardening: access token out of `localStorage` NEW

Covers the code as of 17 Oct 2026 (revised after production-readiness review, and again for in-memory access tokens, section 15) ·Backend: Express on AWS Lambda + MongoDB · Frontend: React SPA (Vite) · File paths are relative to the repository root.

1. The 60-second answer

If an interviewer asks *"How does login work in your app?"*, this is the answer to give, and every later section expands one sentence of it.

> **"We support email/password and 'Sign in with Google or GitHub'. The social login is the OAuth 2.0 Authorization Code flow with PKCE; for Google it's OpenID Connect, so we get a signed ID token.**
>
> The browser hits our backend's `/start` endpoint. The backend generates a random **state** (CSRF protection), a **nonce** (ID-token replay protection) and a **PKCE code verifier**. It stores them in a short-lived, signed, httpOnly cookie and redirects to Google. Google redirects back to our callback with a one-time **authorization code**. We check the state against the cookie, then exchange the code server-to-server using our client secret plus the PKCE verifier.
>
> We verify the ID token's signature against Google's published keys (JWKS), and check the **issuer, audience, expiry and nonce**. Then we map the user by the provider's immutable **subject id**, never by email alone.
>
> The session is handed to our SPA through a **single-use, 60-second ticket** in the URL fragment, which only the browser tab that started the sign-in can redeem. The SPA swaps it for our own JWT access token and httpOnly refresh cookie, exactly like a password login.
>
> From then on every API call goes through one middleware. It verifies the JWT, reloads the user, checks they're active and that the token's **session version** is current (that's how we revoke JWTs), and then role-based authorization decides what they can do."

Three distinctions interviewers love, all of which this app demonstrates:

- **Authentication ≠ authorization.** Login proves *who* you are; `authorize(roles.DEVOPS)` decides *what* you may do.
- **OAuth 2.0 is delegated *authorization*; OpenID Connect is the *authentication* layer on top.** The app uses both. Google sign-in is OIDC (identity). The existing "Connect GitHub" feature is pure OAuth (it gets permission to push to your repos).
- **User identity vs machine identity.** Besides humans, the app authenticates its own AI agent service, GitHub Actions callbacks and AWS workloads, each with a different mechanism (section 4).

2. Prerequisites — what you need before it works

The code is merged and tested, but sign-in buttons only appear once a provider is configured. `GET /api/v1/auth/oauth/providers` returns only providers whose client id *and* secret are set. Here is everything required, in order.

2.1 Google (OpenID Connect)

1. **Google Cloud project.** Go to `console.cloud.google.com`, create a project (one per environment is cleanest: dev / prod).
2. **OAuth consent screen** (APIs & Services → OAuth consent screen / "Google Auth Platform"):
   - User type **External**, app name, support email, logo (optional).
   - **Authorized domains:** your frontend and API domains (e.g. `codemanus.in`). Google requires you to own them.
   - **Privacy policy & Terms URLs:** the app already serves `/legal/privacy` and `/legal/terms`.
   - **Scopes:** only `openid`, `email`, `profile`. These are "non-sensitive", so **no Google security review is required**.
   - While in **Testing** mode only listed test users can sign in. Click **Publish app** for production.
3. **Credentials → Create OAuth client ID → Web application.**
   - **Authorized redirect URIs** (exact, byte-for-byte):
     `http://localhost:4001/api/v1/auth/oauth/google/callback` (local)
     `https://<your-api-host>/api/v1/auth/oauth/google/callback` (prod)
   - "Authorized JavaScript origins" is *not* needed. The browser never talks to Google's token endpoint; our backend does.
4. Copy the **Client ID** and **Client secret** into the backend env (2.3).

2.2 GitHub (OAuth 2.0)

1. GitHub → Settings → Developer settings → **OAuth Apps** → **New OAuth App** (or under your organisation).
2. **Authorization callback URL:** `https://<api-host>/api/v1/auth/oauth/github/callback`. A GitHub OAuth App has exactly **one** callback URL, so create one app per environment.
3. Generate a client secret; copy the Client ID + secret.

> **Why a second GitHub app?**
> The app already has a GitHub OAuth app (`INFRAFLOW_GITHUB_CLIENT_ID`) for *"Connect GitHub"*. It requests `repo workflow` scopes so the platform can push Terraform. Login must not ask for write access to all your repos: that's the **principle of least privilege**. The login app asks only for `read:user user:email`. And since a GitHub OAuth app has a single callback URL, reusing it is impossible anyway.

2.3 Backend environment variables

Variable	Required	What it is
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`	for Google	From 2.1. The secret proves to Google that the code exchange comes from our server (confidential client).
`GITHUB_LOGIN_CLIENT_ID`, `GITHUB_LOGIN_CLIENT_SECRET`	for GitHub	From 2.2. Separate from `INFRAFLOW_GITHUB_*`.

`OAUTH_STATE_SECRET`	**yes, in prod**	HMAC key for the per-attempt transaction cookie. 32+ random bytes. Locally a dev default is used with a warning. **On any deployed environment** (`NODE_ENV=production` or running inside Lambda) Google/GitHub sign-in **stays switched off** until it is set. It fails closed, but only for social login: password login keeps working.
`OAUTH_CALLBACK_BASE_URL`	recommended in prod	Public API URL incl. `/api/v1`. Empty = derived from the request's protocol + host. Set it explicitly behind CloudFront / custom domains so the redirect URI always matches what you registered.
`APP_BASE_URL`	**yes**	The real frontend URL. Default return target after sign-in, and the fallback when the requested origin isn't allow-listed. (Already used by password-reset emails.)
`CLIENT_ORIGIN(S)`	existing	Exact frontend origins. Sign-in returns to the origin it started on only if it's in this list.

Generate secrets with `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`. On Lambda, set them as function environment variables or, better, load them from AWS Secrets Manager / SSM Parameter Store. Never commit them. `IAAS backend/.env.example` documents all of these.

2.4 Platform prerequisites

- **HTTPS in production.** On a deployed server the sign-in cookie is `Secure` and carries the `__Host-` prefix, which browsers accept only over HTTPS. Browsers also only expose `crypto.subtle` (used for the SPA's PKCE challenge) on HTTPS or localhost.
- **No new npm dependency.** Google ID-token verification uses Node's `crypto.createPublicKey({format:'jwk'})` plus the existing `jsonwebtoken`. The Lambda esbuild bundle was rebuilt and verified.
- **MongoDB:** two new collections. `authidentities` has a unique index on `{provider, subject}`. `oauthlogintickets` has a unique `ticketHash` and a **TTL index** on `expiresAt`, so MongoDB deletes expired tickets itself. Mongoose creates both on startup (autoIndex); in locked-down prod, create them explicitly (section 11).
- **Local dev:** open the SPA at an origin in `CLIENT_ORIGINS` (e.g. `http://localhost:5173`). The backend runs on `:4001`.

3. Foundations: OAuth 2.0, OIDC, PKCE, JWT

3.1 The four OAuth roles

Role	Definition	In our Google sign-in
Resource owner	The human who owns the data/account	The person clicking "Continue with Google"
Client	The app asking for access	Our Express backend (a **confidential** client: it can keep a secret)
Authorization server	Authenticates the user, issues codes & tokens	`accounts.google.com` / `oauth2.googleapis.com`
Resource server	API that accepts the access token	Google APIs. For GitHub sign-in: `api.github.com/user`

3.2 OAuth 2.0 vs OpenID Connect

OAuth 2.0 answers "may this app act on my behalf?" and produces an **access token**, an opaque permission slip for an API. It deliberately says nothing about *who* the user is. Using a bare access token as proof of login is a classic bug: a token issued to *another* app could be replayed into yours (the "confused deputy" problem).

OpenID Connect (OIDC) is a thin identity layer on OAuth. Requesting the `openid` scope makes the provider also return an **ID token**, a signed JWT *about the user, addressed to your app*:

```
{ "iss": "https://accounts.google.com",   // who issued it
  "aud": "1234.apps.googleusercontent.com", // MUST be our client id
  "sub": "110169484474386276334",           // stable, unique user id — the identity key
  "email": "person@gmail.com", "email_verified": true,
  "nonce": "Xy7…",                          // echoes the value we sent
  "iat": 1760700000, "exp": 1760703600 }
```

GitHub is **not** an OIDC provider for user login, so for GitHub we use the access token to call `GET /user` and `GET /user/emails`. Because those calls are made by our own server, with a token we just received from GitHub's token endpoint, the confused-deputy problem doesn't apply.

3.3 Grant types: which one and why

Grant	Status	Use
Authorization Code + PKCE	Recommended (OAuth 2.1, RFC 9700)	User login for web, SPA, mobile. **What we use.**
Client Credentials	Current	Machine-to-machine, no user (similar in spirit to our agent JWT).
Device Code	Current	TVs/CLIs without a browser (`gh auth login`).
Refresh Token	Current	Get a new access token without re-login.
Implicit	**Deprecated**	Returned tokens in the URL. Leaky; replaced by Code+PKCE.
Resource Owner Password	**Deprecated**	App collects the user's Google password. Never.

3.4 PKCE in one picture (RFC 7636)

```
code_verifier  = random 32 bytes, base64url        // kept secret by whoever started the flow
code_challenge = BASE64URL( SHA256(code_verifier) )  // sent in the authorize request

authorize?…&code_challenge=E9Mel…&code_challenge_method=S256   → provider remembers challenge
token  POST  code=…&code_verifier=dBjft…                        → provider checks SHA256(verifier) == challenge
```

An attacker who steals the authorization code (via logs, a malicious browser extension, or a mis-registered redirect) cannot redeem it without the verifier, which never left our server. PKCE is required for public clients and recommended for confidential ones too. GitHub added PKCE (S256) support for OAuth apps on 14 July 2025; Google has supported it for years.

We use PKCE twice. Once on the classic leg (backend ↔ provider), and once more on our own leg (SPA ↔ backend) to bind the login ticket to the browser tab that started the sign-in (section 6, steps 2 and 8).

3.5 JWT anatomy, and the classic JWT bugs

`base64url(header).base64url(payload).base64url(signature)`. The header names the algorithm (`alg`) and key id (`kid`). It is **signed, not encrypted**: anyone can read the payload, so never put secrets in it.

- **HS256** = HMAC with a shared secret (our own access/refresh/agent tokens). Whoever can verify can also forge, so the secret must stay inside trusted services.
- **RS256** = RSA signature (Google's ID tokens). Verify with the public key, sign with the private key, which only Google has.
- Bug 1, `alg: none`: the library accepts an unsigned token. Bug 2, **algorithm confusion**: an attacker signs HS256 using the RSA *public* key as the HMAC secret. **Fix for both: pin `algorithms: ['RS256']` on verify.** We do, and a unit test proves both attacks fail.
- Bug 3: decoding (`jwt.decode`) without verifying. We decode only to read `kid`, then verify.

4. Every authentication mechanism in this application

A real system has more than one "login". This is the complete inventory. Being able to walk an interviewer through it shows production experience.

Mechanism	Who is authenticated	How it works here	Key files
Email + password	Human user	bcrypt hash (cost `BCRYPT_ROUNDS`=12) in a Mongoose pre-save hook; `comparePassword` on login; password field `select:false` so it's never returned by accident. Issues our JWTs.	`authController.js` (register/login), `models/User.js`
Sign in with Google / GitHub NEW	Human user	OAuth 2.0 Code + PKCE (OIDC for Google), state cookie, nonce, JWKS verification, identity linking, single-use ticket hand-off. Ends in the same JWTs as password login.	`oauthController.js`, `services/oauthLogin.js`, `services/oauthProviders.js`, `models/AuthIdentity.js`
Session tokens UPDATED	Human user, per request	Access JWT (HS256, 15 min default, `JWT_ACCESS_SECRET`) held **in memory only**, never in `localStorage`, and sent as `Authorization: Bearer`. Refresh JWT (7d, separate secret) in an httpOnly cookie, traded for a new access token on reload and on any 401 (`POST /auth/refresh`). Both carry `sv` (session version) for revocation. Applies to email/password and social login alike. See section 15.	`utils/tokens.js`, `middleware/auth.js`, `src/auth/authClient.ts`, `src/utils/apiClient.ts`
Password reset	Owner of the inbox	32 random bytes emailed; only the **SHA-256** is stored, expiring in 1h. On use: new password, `emailVerified=true`, all sessions revoked (NEW).	`authController.js` forgot/resetPassword, `emailService.js`
Role-based authorization	(authZ, not authN)	Ranked roles superadmin > owner > admin > architect > devops > viewer. `authorize(roles.DEVOPS)` per route; dashboard modules derived from role. Workspace owner self-healing via `ensureWorkspaceOwnerRole`.	`middleware/authorize.js`, `constants/roles.js`, `dashboardModules.js`
"Connect GitHub" (repos)	Delegated access to the user's GitHub	Pure OAuth 2.0 *authorization*: scopes `repo workflow`; state is a JWT bound to our `userId`; GitHub token stored **AES-256-GCM encrypted** (`GITHUB_TOKEN_ENCRYPTION_KEY`); popup + `postMessage` with a nonce-CSP page; scope-drift detection forces reconnect.	`githubController.js`, `githubOAuthService.js`, `models/GitHubConnection.js`

Agent machine JWT	The Python AI agent service	Express mints a 5-minute HS256 JWT per agent run: `aud: "agent-tools"`, `scope: [...]`, `act: userId` (acting-on-behalf-of), separate `AGENT_TOOL_SECRET`. `requireAgentScope('…')` verifies audience + scopes; the Python side verifies with pinned HS256 + audience.	`services/agentIdentity.js`, `middleware/requireAgentScope.js`, `services/agents/app/security.py`
CI callback secret	GitHub Actions job	The Terraform deploy/validate workflows POST results back with a shared bearer secret `DEPLOYMENT_CALLBACK_SECRET`.	`terraformDeployCallbackController.js`, `terraformValidateCallbackController.js`
AWS cross-account role	Our platform → customer AWS	`sts:AssumeRole` into the customer's IAM role with an **ExternalId** (confused-deputy protection), 1-hour temporary credentials; no long-lived customer keys stored.	`services/awsRoleCredentials.js`
GitHub Actions OIDC → AWS	CI workload	Workload identity federation: the workflow gets a GitHub-signed OIDC token, and AWS trusts `token.actions.githubusercontent.com` with conditions `aud = sts.amazonaws.com` and `sub` matching the repo/branch. No AWS keys stored in GitHub.	`deploy/oidc-trust-policy.json`, `pipelineOidcRoleService.js`
Secret reveal throttle	Defence-in-depth	Per-user rate limit (30 / 5 min) on revealing deployment secrets, on top of the global IP limiter.	`middleware/secretRateLimit.js`, `app.js`

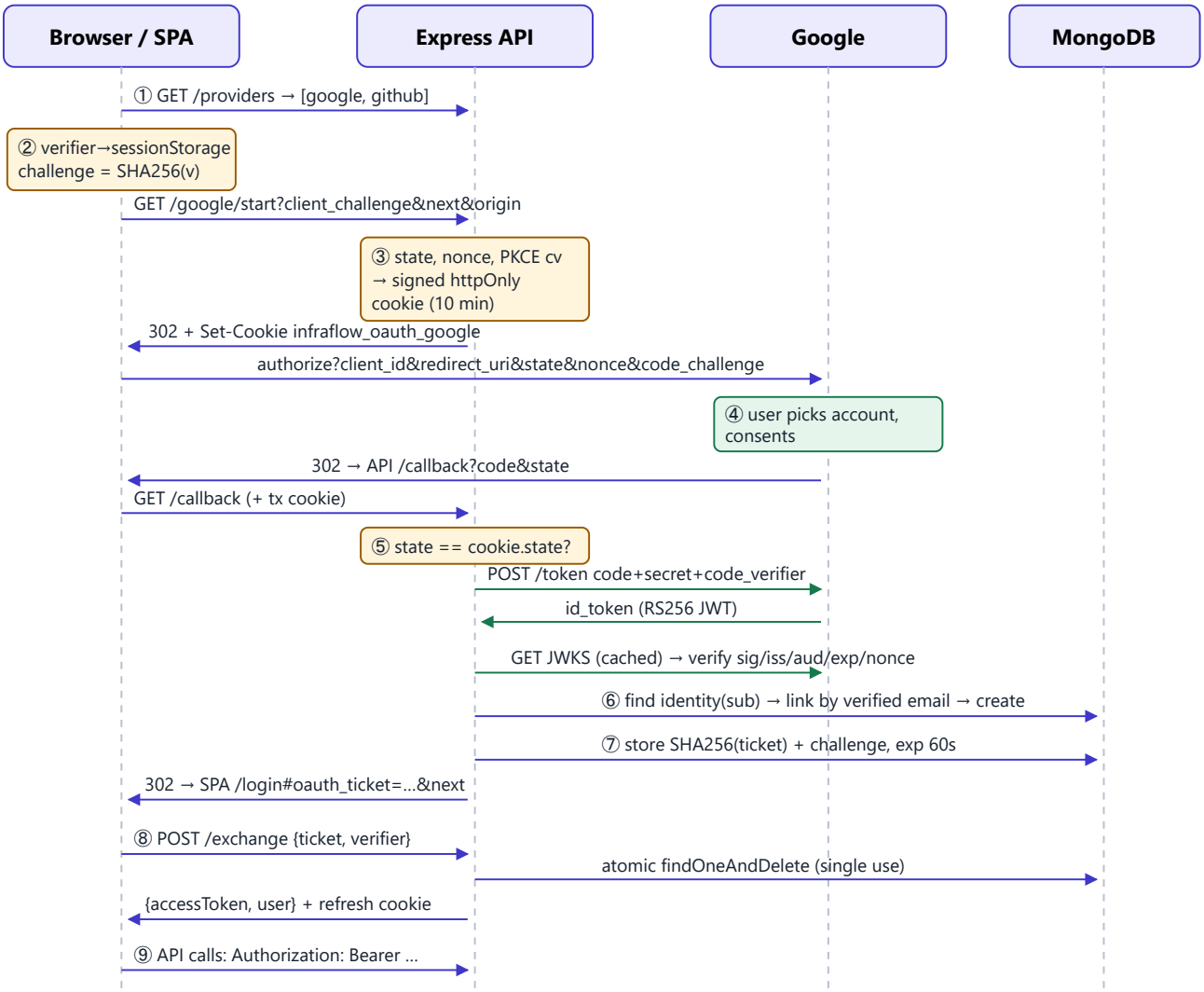
Interview framing

"There are three families: **human authentication** (password, social login, reset), **delegated authorization** (Connect GitHub, where we act on the user's behalf with scoped tokens), and **workload / machine identity** (agent JWTs, CI shared secret, STS AssumeRole with ExternalId, GitHub OIDC federation to AWS). Each uses short-lived credentials, a dedicated secret, and the narrowest scope we could get away with."

5. The new flow — architecture & sequence

Four HTTP legs, all under `/api/v1/auth/oauth`:

Route	Called by	Does
`GET /providers`	Login page (fetch)	Lists configured providers → which buttons render.
`GET /:provider/start`	Browser navigation	Creates state / nonce / PKCE, sets transaction cookie, 302 → provider.
`GET /:provider/callback`	Provider's 302	Validates, exchanges code, verifies identity, links/creates user, mints ticket, 302 → SPA.
`POST /exchange`	SPA (fetch)	Ticket + verifier → our access token + refresh cookie.



Green arrows are back-channel (server-to-server, never visible to the browser). Blue arrows go through the browser. Only the authorization code and the ticket ever appear in a URL. Both are single-use, short-lived and bound (to the PKCE verifier, and to the SPA tab's verifier).

> **Why the backend runs the flow (not the SPA)**
> The SPA could do PKCE alone as a public client, but then the provider tokens and the account-linking decisions would live in JavaScript. Running it server-side makes us a **confidential client** (client secret + PKCE), keeps provider tokens out of the browser entirely (we discard them after reading the profile), and keeps the security-critical decisions (who is this person, may we link, should the old password die) on the server.

6. Code walk-through, step by step

Step ① Which buttons? (`oauthProviders.js`, `AuthPage.tsx`)

Each provider object exposes `isConfigured()`, `authorizeUrl()` and `fetchProfile()`, so the controller never branches on provider. Adding Microsoft or Apple later is one new object. The login page fetches `/providers` and renders only those buttons. Configuration drives the UI, so prod and dev can differ with no code change.

Step ② The SPA starts sign-in (`src/auth/authClient.ts`)

```
src/auth/authClient.ts — startOAuthSignIn()
const verifier  = base64Url(crypto.getRandomValues(new Uint8Array(32)));
const challenge = base64Url(new Uint8Array(await crypto.subtle.digest('SHA-256', new
TextEncoder().encode(verifier))));
window.sessionStorage.setItem(OAUTH_VERIFIER_KEY, verifier);   // stays in THIS tab
const params = new URLSearchParams({ client_challenge: challenge, next, origin: window.location.origin });
window.location.assign(`${API_BASE_URL}/auth/oauth/${providerId}/start?${params}`);
```

Why `sessionStorage`? It is per-tab and per-origin, and it survives the full-page redirects to Google and back. Only this tab can later prove it started the flow. **Why a full navigation (not fetch)?** The user has to see Google's consent page, and the cookie set by `/start` must be a first-party cookie on the API domain.

Step ③ `/start` creates the transaction (`oauthController.js`, `oauthLogin.js`)

```
IAAS backend/src/services/oauthLogin.js — createOAuthTransaction()
const tx = { typ: 'oauth_tx', p: providerId,
  state: randomToken(24),   // CSRF: must come back unchanged
  nonce: randomToken(24),   // goes into Google's id_token
  cv: codeVerifier,         // PKCE verifier for the provider leg
  cc: clientChallenge,      // SPA's challenge for the ticket leg
  rt: returnTo, ao: appOrigin };
cookieValue = jwt.sign(tx, env.OAUTH_STATE_SECRET, { expiresIn: 600 });

cookie name & options
name:    (IS_DEPLOYED ? '__Host-' : '') + 'infraflow_oauth_google'
options: { httpOnly: true, secure: IS_DEPLOYED, sameSite: 'lax', path: '/', maxAge: 600_000 }
// IS_DEPLOYED = NODE_ENV==='production' || running inside Lambda (AWS_LAMBDA_FUNCTION_NAME)
```

- **Why a signed cookie, not a server session?** We run on Lambda, with no sticky memory between `/start` and `/callback`. A signed, httpOnly, 10-minute cookie is a stateless "session for one sign-in". Signing stops tampering; httpOnly stops page JavaScript reading the verifier.
- **Why SameSite=Lax, not Strict?** The callback arrives as a *cross-site top-level GET* redirect from accounts.google.com. Lax cookies are sent on that; Strict ones are not, and sign-in would always fail.
- **Why the state must live in a cookie** (and not just be a signed value in the URL): a signed state proves *we* issued it, not that *this browser* started the flow. Binding it to the browser is what stops **login CSRF**. The existing GitHub-connect flow uses a signed state bound to the logged-in `userId` instead, which works there because that user is already logged in.
- **Why `__Host-`?** The browser then enforces `Secure`, `Path=/` and *no* `Domain` attribute, so a sibling subdomain (a compromised `blog.codemanus.in`, say) cannot plant or overwrite this cookie ("cookie tossing"). Browsers reject the prefix over plain http, hence deployed-only.
- **Why not just check `NODE_ENV`?** Someone will eventually forget to set it. Running inside Lambda (`AWS_LAMBDA_FUNCTION_NAME`, which AWS always sets) is a signal that can't be forgotten. The same flag drives the `Secure` cookie and the origin rules below.

- `safeReturnTo()` accepts only same-app relative paths (rejects `//evil.com` and `/\evil.com`). `allowedAppOrigin()` is an **exact-match** allowlist. On a deployed server it also drops every `http:` and loopback origin, because the existing `env.js` always merges `http://localhost:5173` into `CLIENT_ORIGINS`. A request without a valid `client_challenge` (43 base64url chars) is sent back to the login page with an error code.

Step ④ The user is at Google

We send `scope=openid email profile`, `response_type=code`, `code_challenge_method=S256` and `prompt=select_account` (lets people with several Google accounts choose). For GitHub: `scope=read:user user:email`. We never see the user's Google password.

Step ⑤ `/callback`: validate, exchange, verify

```
IAAS backend/src/controllers/oauthController.js — oauthLoginCallback()
res.clearCookie(cookieName, clearOptions);            // single use, whatever happens next
const tx = readOAuthTransaction({ cookieValue, providerId, state });  // timing-safe state compare
if (error === 'access_denied') throw oauthError(400, 'cancelled', 'User cancelled at the provider.');
const profile = await provider.fetchProfile({ code, redirectUri, codeVerifier: tx.codeVerifier, nonce: tx.nonce
});
```

The callback is a **browser navigation**, so every outcome, including errors, ends in a redirect back to `/login?oauth_error=<code>`, never a raw JSON error page. `/start` behaves the same way. The redirect carries only a short **code**, never text (section 6.1). The `redirect_uri` sent in the token exchange must equal the one sent to `/authorize`; both come from the same `oauthCallbackUrl()`.

Every call to Google or GitHub has a 10-second timeout (`providerFetch()` uses `AbortSignal.timeout`). Without it, a slow or hanging provider would hold the Lambda until the function's own timeout, which is billed time and a stuck user. A timeout or network failure becomes a clean `provider_error`.

```
IAAS backend/src/services/oauthProviders.js — verifyGoogleIdToken()
const decoded   = jwt.decode(idToken, { complete: true });          // only to read header.kid
const publicKey = await googlePublicKey(decoded.header.kid, now);    // JWKS, cached per Cache-Control
claims = jwt.verify(idToken, publicKey, {
  algorithms: ['RS256'],          // pinned — blocks alg:none & HS256 confusion
  audience: clientId,             // token was issued to US
  issuer: ['https://accounts.google.com', 'accounts.google.com'],
  clockTolerance: 30 });          // small clock skew allowance
if (!safeEqual(claims.nonce, nonce)) throw …;                        // bound to this attempt
```

JWKS caching and key rotation. Keys are cached for Google's `max-age`. An *unknown* `kid` forces one refetch, so when Google rotates keys sign-in keeps working without waiting for cache expiry. Public keys come from `crypto.createPublicKey({ key: jwk, format: 'jwk' })`, the same thing libraries like `jose` do internally.

GitHub has no ID token. We exchange the code (with `code_verifier`), then call `/user` (id, login, name, avatar) and `/user/emails`. We choose the **primary verified** email, else any verified one, else we report "no verified email". The GitHub token is used for those two calls and discarded; we don't store tokens we don't need (data minimisation).

Step ⑥ Who is this? (section 7 covers the reasoning)

```
IAAS backend/src/services/oauthLogin.js — resolveOAuthUser()
1. AuthIdentity.findOne({ provider, subject })           → known person: sign in
2. require profile.emailVerified                           → else 403
3. User.findOne({ email })                                 → link (pre-hijack defence)
4. otherwise User.create(owner, no password, emailVerified) + Workspace + AuthIdentity
   E11000 duplicate key from a concurrent callback         → retry once (finds the winner's rows)
```

Step ⑦ The single-use ticket

```
issueLoginTicket()  — models/OAuthLoginTicket.js, its own collection
const ticket = randomToken(32);                          // 256 bits — unguessable
OAuthLoginTicket.create({
  ticketHash: sha256(ticket),                             // DB leak ≠ usable tickets
  clientChallenge: tx.clientChallenge,                    // bound to the SPA tab
  userId, notice,                                         // notice: 'password_removed' | null
  expiresAt: now + 60s });                                // TTL index deletes it automatically
res.redirect(`${appOrigin}/login#oauth_ticket=${ticket}&next=${returnTo}`);
```

Why a separate collection rather than fields on `User`? One row per sign-in means two tabs or two devices signing in at the same moment each get their own ticket, rather than the second overwriting the first. The TTL index cleans up abandoned tickets. MongoDB's TTL sweeper only runs about once a minute, so the redeem query still checks `expiresAt` itself.

Why not put the JWT in the redirect URL? URLs end up in browser history, proxy/CDN logs, `Referer` headers and screenshots. The **fragment** (`#…`) is never sent to any server. Even so we send only a short-lived, single-use, browser-bound ticket, never the session itself. Same reasoning as the password-reset token: **store hashes of bearer secrets, not the secrets**.

Step ⑧ The SPA redeems the ticket (`AuthPage.tsx` → `/exchange`)

```
src/auth/AuthPage.tsx (effect on mount)
const ticket = new URLSearchParams(location.hash.slice(1)).get('oauth_ticket');
history.replaceState(null, '', location.pathname);   // scrub URL + history BEFORE any await
await completeOAuthSignIn(ticket);                    // sends {ticket, verifier from sessionStorage}

IAAS backend/src/services/oauthLogin.js — redeemLoginTicket()
OAuthLoginTicket.findOneAndDelete({
  ticketHash: sha256(ticket),
  clientChallenge: sha256b64url(verifier),
  expiresAt: { $gt: new Date() } });   // then load the user, check status, return { user, notice }
```

One atomic **find-and-delete**: of two concurrent redemptions only one can match. That is single-use without a separate lock. Scrubbing the URL synchronously also makes React StrictMode's double-invoked effect harmless: the second run finds no ticket. The response matches `/auth/login`'s, plus an optional `notice`, so the rest of the app can't tell the two apart. If `notice` is `password_removed`, the page stops and explains before continuing (section 7.3).

6.1 Failure handling & observability

Codes, not text. Every sign-in error is raised as `oauthError(status, code, internalMessage)`. The browser is redirected with only the code. The login page alone owns the wording:

Code	When	What the user reads
`cancelled`	User pressed Cancel at the provider	Sign-in was cancelled.
`state_invalid`	Cookie missing/expired/tampered, state mismatch	…expired or was started in a different browser…
`email_unverified`	Provider has no verified email	…verify one there, then try again.
`account_inactive`	Disabled account	…contact your workspace administrator.
`not_enabled`	Unknown or unconfigured provider	That sign-in method is not available right now.

`provider_error`	Token exchange, ID-token, JWKS, timeout, provider-side error	We couldn't complete sign-in with the provider…
anything else	Unexpected	Sign-in failed. Please try again.

> **Why this matters (the bug it fixed)**
> The first version put the error *message* in the URL and the page displayed it. React escapes text, so it wasn't XSS. But anyone could mail out `codemanus.in/login?oauth_error=Your account is locked, call +91…` and *your own login page* would show the attacker's words: **content spoofing**, a phishing aid. Mapping fixed codes to fixed text closes it, and a browser test proves crafted text is never displayed.

Logging. Most failures happen before there is a user, so they can't go to the audit log. Every failure writes one structured log line: `{ provider, code, statusCode, reason, ip }`. It never contains the code, state, tokens or cookie. Levels are chosen for alerting: a user pressing Cancel is `info`, anything we refused (bad state, unverified email) is `warn`, and our own or the provider's failure is `error`. A spike of `warn` lines with `state_invalid` suggests an attack; a spike of `error` lines suggests a provider outage or misconfiguration.

Step ⑨ Every API request afterwards (`middleware/auth.js`)

```
verifyAccessToken(bearer)          // signature + expiry; bad/expired → 401 (was a 500 before this change)
user = User.findById(payload.sub)  // fresh DB read: disabled users are cut off immediately
user.status === 'active'
assertSessionCurrent(payload, user) // payload.sv === user.sessionVersion  (NEW)
authorize(roles.DEVOPS)             // role rank check, per route
```

7. Account linking & pre-account hijacking

This is the most subtle part of social login and the best interview material, because many production apps get it wrong.

7.1 Identify by `subject`, not by email

The new `AuthIdentity` collection has one row per external login, keyed by `{provider, subject}` (unique index). Google's `sub` and GitHub's numeric `id` never change and are never reused. Email addresses *can* change, get recycled (a company deletes `alice@corp.com`, hires another Alice), or be changed on the provider side. Keying by email would let the next owner of an address inherit the previous owner's account.

A separate collection (not an array on `User`) gives a clean unique index, mirrors the existing `GitHubConnection` pattern, and lets one user link several providers.

7.2 Only trust *verified* emails

When there is no identity yet, we fall back to email to link an existing account, but only if the provider says the email is verified (`email_verified: true` for Google; `verified` in GitHub's `/user/emails`). Otherwise an attacker could add `victim@company.com` as an unverified address on some provider and sign straight into the victim's account.

7.3 The pre-account-hijacking attack

The attack (Sudhodanan & Paverd, 2022: the "classic–federated merge")

1. The attacker registers `victim@gmail.com` with a password *before* the victim ever uses the app. Our registration never verified email, so this succeeds.
2. Later the real victim clicks "Continue with Google". Google proves they own `victim@gmail.com`, and a naive app links Google to the existing account.
3. Now **both** can log in: the victim with Google, the attacker with the password they set. The attacker silently watches or uses everything the victim does, such as AWS connections and deployments.

Our defence

IAAS backend/src/services/oauthLogin.js

```
if (existing.password && !existing.emailVerified) {
  existing.password = undefined;                                  // unproven password dies
  existing.sessionVersion = (existing.sessionVersion ?? 0) + 1; // every old JWT dies
  passwordCleared = true;                                          // audited
}
existing.emailVerified = true;
```

The person who just proved control of the inbox wins. If the account owner was legitimate all along, they simply use **Forgot password** (which also proves inbox control) to set a password again. **They are told immediately:** the ticket carries `notice: 'password_removed'`, and the login page stops on a notice panel ("You're signed in. One thing changed on your account…") with a Continue button, rather than redirecting straight to the dashboard. A later login attempt with the old password also gets an explicit message pointing them to Forgot password. This mirrors how Firebase Auth treats a trusted provider signing into an unverified email/password account. If the email *was* already verified (via an earlier password reset or OAuth login), nothing is cleared: both methods keep working.

The end-to-end tests reproduce this exact attack: attacker registers, victim signs in with Google, attacker's live session returns 401 "Session has been revoked", the attacker's password no longer works, and the victim sees the notice.

> **Before you launch: this touches every existing user**
> Registration never verified email, so **every account that exists today has** `emailVerified=false`. The first time any existing password user clicks “Continue with Google/GitHub”, their password is removed (they see the notice, and Forgot password restores it). If that is too disruptive, send verification emails to existing users *before* enabling the buttons. Verified accounts keep their password when linked.

7.4 Brand-new users

New users get role `owner` and their own workspace (same as `/register`), **no password** (the schema’s `required` was removed and `comparePassword` returns `false` when there is none), `emailVerified: true`, and a legal-consent record: the login page states that continuing with a provider accepts the Terms and Privacy Policy, in place of the register form’s checkbox. If a concurrent duplicate callback wins the race, the half-created user is deleted and the flow retries once (MongoDB duplicate-key error `11000`).

8. Sessions, JWTs & revocation

8.1 What a session is in this app

Token	Lifetime	Stored	Sent as	Claims
Access JWT (HS256)	15 min default (`JWT_ACCESS_EXPIRES_IN`; was 6 h)	SPA memory only (a module variable in `src/auth/authClient.ts`)	`Authorization: Bearer`	`sub, role, workspace, sv`
Refresh JWT (HS256, own secret)	7 d	httpOnly cookie on API domain; `SameSite=None; Secure` when deployed, `Lax` locally	Cookie → `POST /auth/refresh` on every reload / new tab and on any 401	`sub, type:'refresh', sv`

Before 17 Oct 2026 the access token lived in `localStorage` for 6 h and the refresh cookie was never used. Section 15 describes the change.

8.2 The stateless-JWT revocation problem, and our fix

A JWT is valid until it expires: the server keeps no list of them. That's why JWTs scale, and also why "log out everywhere", "I changed my password" or "we caught an attacker" cannot simply delete a session. The usual fixes:

Approach	Cost	Used here?
Very short access tokens + refresh rotation	More refresh traffic, complexity	**Yes** for short tokens: 15 min + silent refresh (section 15). Refresh-token rotation: not yet (section 12.2).
Denylist of revoked token ids (`jti`) in Redis	Lookup per request, extra infra	No
Per-user version counter in the token	One integer compare; we already load the user each request	**Yes** NEW

`User.sessionVersion` is embedded in every token as `sv`. `assertSessionCurrent()` runs in `requireAuth`, `/auth/refresh` and the GitHub-connect entry. Incrementing the counter instantly invalidates **every** token issued before. It is bumped on a password reset and on the pre-hijack link. Tokens minted before this change have no `sv` and count as version 0, which matches every existing user, so **nobody was logged out by the deploy**.

Also fixed: an expired or tampered JWT used to throw a raw `JsonWebTokenError`, which the error handler turned into a **500**. It is now a proper **401**. `verifyRefreshToken` also checks `type === 'refresh'`.

8.3 Secret separation, a deliberate pattern in this codebase

`JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `AGENT_TOOL_SECRET`, `GITHUB_TOKEN_ENCRYPTION_KEY`, `DEPLOYMENT_CALLBACK_SECRET` and now `OAUTH_STATE_SECRET` are all different. One key per purpose limits the blast radius of a leak, and lets you rotate one without logging everyone out or bricking encrypted data. Rotating `OAUTH_STATE_SECRET` only abandons sign-ins in flight at that moment (at most 10 minutes' worth).

9. Threat model: each defence and the attack it stops

Attack	What would happen	Defence in this implementation
Login CSRF (provider leg)	Attacker feeds victim a callback URL with the *attacker's* code; victim is silently logged into the attacker's account and uploads their AWS details into it.	`state` stored in the victim's own httpOnly cookie; callback rejects mismatch (timing-safe compare).
Login CSRF (ticket leg)	Attacker sends `/login#oauth_ticket=<attacker's ticket>`.	Ticket bound to the challenge of the tab that started the flow; victim's tab has no matching verifier → refused.
Authorization-code interception	Stolen code redeemed by attacker.	PKCE S256 + client secret; code is single-use at the provider.
ID-token replay / injection	An id_token captured elsewhere replayed into our callback.	`nonce` from the browser-bound cookie must match the token's nonce.
Token substitution (confused deputy)	A valid Google token issued to *another* app used to log into ours.	`aud` must equal our client id; `iss` must be Google.
`alg:none` / RS→HS confusion	Forged ID token accepted.	`algorithms: ['RS256']` pinned; unit-tested.
Expired / foreign-key token	Old or self-signed token accepted.	`exp` enforced (30 s skew); `kid` must be in Google's JWKS.
Open redirect → ticket theft	Callback redirects to `attacker.vercel.app` with a valid ticket.	Exact-match origin allowlist; `*.vercel.app` is **not** accepted as a redirect target (anyone can deploy there). `next` limited to relative paths. CORS now uses the same exact-match rule (section 15).
XSS steals the session NEW	Injected script reads the access token from `localStorage` and replays it elsewhere for hours.	Access token kept in memory only, 15 min lifetime; the refresh token is in an httpOnly cookie that scripts can't read (section 15).
Another site uses the refresh cookie NEW	A page on `attacker.cloudfront.net` calls `/auth/refresh` with the visitor's `SameSite=None` cookie and reads back their access token.	Credentialed CORS only for exact, listed origins. On a deployed API, no http or loopback origins either (`utils/cors.js`).
Token leakage via URL	Session token in history, logs, Referer.	Only a 60 s single-use ticket, in the fragment, scrubbed with `replaceState` immediately.
Ticket replay / race	Ticket redeemed twice.	Atomic `findOneAndDelete`; only the hash is stored; TTL index removes leftovers.
Content spoofing via error text	Crafted `/login?oauth_error=…` link makes our page display an attacker's message ("call this number").	URL carries only a code; the page maps codes to fixed wording. Browser-tested.
Slow / hanging provider	Lambda held until its own timeout; cost and stuck users.	10 s `AbortSignal.timeout` on every provider call → clean `provider_error`.

Cookie tossing from a subdomain	A sibling subdomain plants a sign-in cookie for our API.	`__Host-` cookie prefix on deployed servers.
Deploying without the state secret	Sign-in cookies signed with a publicly known key.	Deployed servers switch social sign-in off until `OAUTH_STATE_SECRET` is set (fail closed, password login unaffected).
Redirect to localhost on a deployed server	Ticket delivered to whatever listens on the victim's localhost.	Deployed servers drop `http:` and loopback origins from the redirect allowlist.
Account takeover via unverified email	Attacker adds victim's email unverified at provider.	Link/create only when `email_verified` / GitHub `verified`.
Pre-account hijacking	Attacker pre-registers victim's email with a password.	Unverified password cleared + `sessionVersion` bump on link.
Email recycling / change	New owner of an address inherits the account.	Identity keyed by immutable `subject`.
Over-broad consent	Login grants write access to all repos.	Separate GitHub login app with `read:user user:email` only.
Disabled user keeps access	Ban not effective until token expiry.	Every request reloads the user and checks `status`; callback and exchange also check.
Stolen provider token at rest	DB leak exposes Google/GitHub tokens.	Login tokens are never stored. (Repo-connect tokens are AES-256-GCM encrypted.)
Forensics / abuse detection	No trail.	Audit log: `auth.oauth.registered | login | linked` (+ `passwordCleared`, provider, IP). Every failure: one structured `[oauth]` log line at info/warn/error.

10. Testing strategy

Three layers, from fastest to most realistic. **All three live in the repository** and all passed when this guide was last updated (backend: 78/78 with MongoDB; 72 + 1 skipped without; frontend 57/57; browser check 21/21).

10.1 Unit tests: `IAAS backend/test/oauthLogin.test.js` (22 tests, `npm test`)

- PKCE matches the **RFC 7636 Appendix B test vector** (`dBjftJeZ…` → `E9Melhoa…`). Testing against a published vector proves the base64url/SHA-256 details rather than just round-tripping our own code.
- `safeReturnTo` rejects `//evil`, `/\evil` and absolute URLs. `allowedAppOrigin` rejects `attacker.vercel.app` and `javascript:`, and on a deployed server also rejects `http://localhost` / `127.0.0.1` even though `CLIENT_ORIGINS` lists them. The cookie gets the `__Host-` prefix only when deployed.
- Transaction cookie: round-trip; rejects wrong state, missing/tampered cookie, and a cookie for another provider, each with code `state_invalid`.
- Google ID token, verified with a locally generated RSA key served as a fake JWKS: accepts a valid token and caches keys; rejects wrong `aud`, wrong nonce, expired, foreign signing key, unknown `kid`, `alg:none` and HS256 confusion.
- Provider calls carry an abort signal; a timeout becomes a 502 `provider_error` rather than a hang.
- Account resolution with mocked Mongoose (same pattern as `credits.test.js`): known subject → login; unverified email → `email_unverified`; unverified password account → password cleared + version bumped; verified account → untouched; disabled → `account_inactive`; new user → owner, no password, workspace + identity.
- Tickets: two sign-ins create two rows (no overwrite), only the hash is stored, ~60 s expiry, notice carried; redeem is one atomic find-and-delete on hash + challenge + expiry; used/expired → `ticket_invalid`; a user disabled in between is refused.

10.2 Integration: `IAAS backend/test/oauthFlow.integration.test.js` (6 tests, real HTTP + real MongoDB)

Mounts the real auth router on Express with Google's token and JWKS endpoints faked *in-process*. The fake token endpoint enforces PKCE exactly as Google does (`SHA256(code_verifier) == code_challenge`). It creates its own uniquely named database and drops it afterwards. It **skips itself** unless `OAUTH_IT_MONGODB_URI` is set, so plain `npm test` and CI stay green without a database.

```
OAUTH_IT_MONGODB_URI=mongodb://127.0.0.1:27017 npm test      # IAAS backend/
```

Covers: brand-new user (ticket only in the fragment, wrong verifier refused, single use, tickets deleted); returning user with no duplicate; **two sign-ins in flight for the same person both succeed**; the federated-only password-login message; the full **pre-hijack attack** (attacker's session and password die, victim gets `notice: password_removed`, audit shows `passwordCleared`); and failures arriving as **codes only** (`provider_error`, `cancelled`, `state_invalid`, `not_enabled`).

10.3 Real browser: `npm run e2e:oauth-browser` (21 checks)

`IAAS backend/scripts/e2e/oauth-browser-check.mjs` runs the real backend app in-process (fake Google again) and drives headless Chrome over the DevTools protocol against the real SPA served by Vite. Requests to `accounts.google.com` are intercepted to act as the consent screen. It checks the click → `/start` → "Google" → callback → `/login#ticket` → exchange → **dashboard** round trip; token stored, verifier consumed, URL scrubbed; error codes rendered with fixed wording; **crafted text in `?oauth_error=` never displayed**; a planted ticket refused; the **"password removed" notice** shown before continuing; and both UI experiences. Since section 15 it also checks

that **no JWT is ever in** `localStorage` / `sessionStorage` (Google and password login), that a **reload stays signed in** through the refresh cookie, that an **expired access token is refreshed and the request retried**, and that a reload after logout lands on `/login`. Run `npm run dev` at the repo root first (the script uses `http://127.0.0.1:5173`); set `E2E_SCREENSHOT_DIR` to keep screenshots. Restart Vite if you edited `authClient.ts` while it was running: hot reload can leave two copies of the module, each with its own in-memory token.

Also verified by hand: deployed mode (simulated Lambda) without `OAUTH_STATE_SECRET` lists no providers and `/start` returns `not_enabled`; with it, the cookie is `__Host-…; HttpOnly; Secure; SameSite=Lax`. The Lambda esbuild bundle builds.

10.4 Testing with real Google locally

1. Register `http://127.0.0.1:4001/api/v1/auth/oauth/google/callback` on a dev OAuth client (for GitHub: a separate dev OAuth app with `…/auth/oauth/github/callback`); add yourself as a test user. Use `127.0.0.1`, not `localhost`: in dev the SPA calls the API at `127.0.0.1:4001`, and the backend derives the callback from that host.
2. Set `GOOGLE_CLIENT_ID/SECRET` (and/or `GITHUB_LOGIN_CLIENT_ID/SECRET`) and `OAUTH_STATE_SECRET` in `IAAS backend/.env`. Leave `OAUTH_CALLBACK_BASE_URL` empty locally.
3. `run-backend-dev` + `run-frontend-dev`, open `http://127.0.0.1:5173/login`. A dev page on `localhost` redirects itself to `127.0.0.1` so the refresh cookie is same-site (section 15.4).
4. **Common error:** `redirect_uri_mismatch` means the URI sent differs from the registered one by even a trailing slash, `127.0.0.1` vs `localhost`, or http vs https. Set `OAUTH_CALLBACK_BASE_URL` to pin it.

11. Production rollout checklist

#	Item	Why
1	Create prod Google OAuth client + prod GitHub login app; register the prod callback URLs exactly.	Separate clients per environment; dev secrets never touch prod.
2	Publish the Google consent screen (Testing → In production).	In Testing mode only listed test users can sign in.
3	Set `GOOGLE_*`, `GITHUB_LOGIN_*`, `OAUTH_STATE_SECRET`, `OAUTH_CALLBACK_BASE_URL`, correct `APP_BASE_URL` on the Lambda (ideally from Secrets Manager).	Missing `APP_BASE_URL` would send users to localhost after sign-in.
4	Set `NODE_ENV=production` (Lambda is detected automatically either way).	Belt and braces: `Secure` + `__Host-` cookies and the https-only redirect allowlist key off "deployed".
5	If autoIndex is off, create: `db.authidentities.createIndex({provider:1, subject:1}, {unique:true})`, `db.oauthlogintickets.createIndex({ticketHash:1}, {unique:true})`, `db.oauthlogintickets.createIndex({expiresAt:1}, {expireAfterSeconds:0})`.	Uniqueness makes concurrent first logins safe; TTL cleans abandoned tickets.
6	Make sure every frontend origin users visit (e.g. `https://codemanus.in`, preview deployments) is in `CLIENT_ORIGIN`.	Otherwise sign-in returns to `APP_BASE_URL`; if that's a different origin, the tab's verifier isn't there and the exchange is refused. Since section 15, CORS has no wildcards either: an unlisted frontend can't call the API at all.
7	Smoke test: sign in with Google and GitHub; cancel at the consent screen; sign in with a GitHub account whose email is unverified.	Exercises success, user-cancel, and the verified-email gate.
8	Monitor (CloudWatch Logs metric filters): `[oauth] sign-in failed` (error: provider outage / misconfig), `[oauth] sign-in rejected` with `state_invalid` (possible attack), audit `auth.oauth.*` with `passwordCleared:true`.	Each failure is one structured line with a code; alert on spikes.
9	Decide on existing users (section 7.3): accept that a first social sign-in removes an unverified password (users see a notice), or send verification emails before enabling the buttons.	Every current account is unverified.
10	Rotation plan: rotate client secrets yearly or on suspicion (Google allows two active secrets for zero-downtime rotation).	Standard secret hygiene.
11	Before each release touching auth: `OAUTH_IT_MONGODB_URI=… npm test` and `npm run e2e:oauth-browser`.	Integration and browser layers aren't in CI (no DB/browser there).
12	**First real-provider test** (never done yet): one manual sign-in each with real Google and real GitHub in dev, then in prod.	Automated tests use a faithful fake; only the real thing proves your client config and GitHub's PKCE handling.

Cross-site cookie caveat (now matters, see section 15.7)

The SPA (CloudFront / codemanus.in) and the API (execute-api) are different sites. Since the access token moved to memory, the SPA **depends on** `POST /auth/refresh` after every reload, so the refresh cookie is now `SameSite=None; Secure` when deployed. Browsers that block third-party cookies (Safari by default) still won't send it, and their users are signed out on every reload. Serving the API under the same site (e.g. `api.codemanus.in`, or a CloudFront `/api/*` route) fixes this properly.

12. Production-readiness review & known gaps

12.1 What a self-review of the first version caught (and how it was fixed)

After the feature worked end to end, it was reviewed as a production change. Being able to say *"here is what I found in my own code"* is a strong interview signal.

Finding	Why it mattered	Fix
Error messages travelled in the URL and were displayed	Content spoofing: anyone could make our login page show their text.	Codes only; the page owns the wording (6.1). Browser-tested.
No timeouts on Google/GitHub calls	A slow provider holds the Lambda until its own timeout.	`AbortSignal.timeout(10 s)` on every call → `provider_error`.
`localhost` accepted as redirect target in prod	`env.js` always merges dev origins into `CLIENT_ORIGINS`.	Deployed servers drop http/loopback origins.
Silent password removal	Users found out only when their password failed later.	One-time notice carried on the ticket; page stops and explains.
4xx failures not logged	No visibility into attacks or misconfiguration.	One structured line per failure, levelled info/warn/error.
Default state secret only warned in prod	Cookies signed with a public key.	Social sign-in disabled on deployed servers until set.
One pending ticket per user (fields on `User`)	Two devices signing in together: the first failed.	Own collection, one row per sign-in, TTL index.
Cookie without `__Host-`	Subdomain cookie tossing.	Prefix on deployed servers.
E2E tests only on a laptop	Nobody could rerun them.	Integration test + browser script committed.

12.2 Remaining gaps in the wider app (not changed by this work)

Interviewers often ask "what would you do differently?". Honest, prioritised answers score far better than claiming perfection. These were found while reading the code for this work and were **not** changed, to keep this change focused.

Gap	Risk	Fix
~~Access token in `localStorage`~~ **FIXED**	~~Any XSS can read and exfiltrate it (6 h validity).~~	Done (section 15): in memory only, 15 min, silent refresh from the httpOnly cookie. Next step is a backend-for-frontend or a same-site API domain.
Refresh token not rotated	7-day bearer with no reuse detection. It is now used on every reload (section 15).	Rotate the refresh token on every use and store a family id to detect reuse (OAuth 2.1 / RFC 9700 guidance). Access tokens are already 15 min.
GitHub-connect passes the access token in a URL (`githubOAuthUrl` in `src/github/githubApi.ts`, `?token=`)	JWT in history and server/CDN logs (now valid 15 min at most; the `console.log` that printed it was removed).	Reuse the ticket pattern from this work, or a short-lived single-purpose "connect intent" token.

CI callback secret compared with `!==`	Theoretical timing side channel.	`crypto.timingSafeEqual`; better, verify a GitHub OIDC token from the workflow instead of a shared secret.
No email verification at registration	Root cause of the pre-hijack risk; typo'd emails.	Send a verification link; set `emailVerified`; restrict risky actions until verified.
Account enumeration	`forgot-password` returns 404 for unknown emails; login reveals federated accounts.	Uniform "if an account exists, we sent an email" responses.
No MFA	Password-only accounts are phishable.	TOTP / WebAuthn passkeys; or encourage social login (which inherits Google's MFA).
Only the global rate limit on `/auth/login`	Credential stuffing at 600 req / 15 min per IP.	Per-account + per-IP login throttling with backoff; breached-password checks.
Symmetric HS256 user JWTs	Any service that verifies tokens could also mint them.	RS256/EdDSA with a published JWKS once multiple services verify user tokens.
No self-service "linked accounts" UI	Users can't see or unlink providers.	Settings page listing `AuthIdentity` rows with unlink (never unlink the last login method).

13. Interview questions & model answers

Q1. What's the difference between OAuth 2.0 and OpenID Connect?

OAuth 2.0 is a delegation framework: it gives an app an access token to call an API on a user's behalf, and says nothing standard about who the user is. OIDC adds an identity layer: with the `openid` scope the provider also returns a signed ID token (JWT) with `iss`, `aud`, `sub`, `exp` and `nonce`, addressed to my client. Login should use OIDC. Our Google sign-in is OIDC; our "Connect GitHub" is plain OAuth for repo access.

Q2. Walk me through the Authorization Code flow with PKCE.

Client makes a random verifier and sends its SHA-256 (the challenge) plus `state` in the authorize redirect. The user authenticates and consents at the provider, which redirects back with a short-lived code. The client checks state, then POSTs the code, its client secret and the verifier to the token endpoint. The provider checks SHA256(verifier) equals the stored challenge and returns tokens. A stolen code is useless without the verifier.

Q3. Why is the Implicit flow deprecated?

It returned access tokens directly in the redirect URL fragment, exposed to history, referrers, extensions and injection, with no way to bind them to the client. Code + PKCE gives SPAs the same capability without tokens in URLs.

Q4. What are `state` and `nonce`, and how are they different?

`state` protects the *redirect*: an unguessable value tied to the user's browser (we keep it in an httpOnly cookie) that must come back unchanged, which prevents login CSRF. `nonce` protects the *ID token*: it is embedded in the token by the provider, and we require it to match, which prevents replaying an ID token from another session. PKCE protects the *code*.

Q5. How do you validate an ID token?

Read `kid` from the header; fetch the provider's JWKS (cached, refetch on unknown kid); verify the signature with an **allow-listed algorithm** (RS256); check `iss` is the provider, `aud` is my client id, `exp` / `iat` with small skew, and `nonce` matches. Then require `email_verified` before trusting the email.

Q6. Why key users by `sub` instead of email?

`sub` is stable and unique per provider; emails change and get recycled. Keying on email lets a future owner of an address take over the account. We store `{provider, subject}` with a unique index and use email only for a one-time, verified-email link.

Q7. What is pre-account hijacking and how did you prevent it?

An attacker registers the victim's email with a password before the victim joins. When the victim later uses Google, a naive app merges the accounts and the attacker's password still works. Because our registration doesn't verify email, on linking we clear any password on an unverified account and bump the session version, which revokes the attacker's live JWTs. The legitimate owner can re-set a password through the email-based reset.

Q8. JWTs are stateless — how do you log someone out everywhere?

Options are short expiry plus rotating refresh tokens, a denylist of `jti`s, or a per-user version. We embed `sv` (session version) in every token, and the auth middleware, which already loads the user per request for status checks, compares it. Incrementing it revokes every outstanding token at the cost of one integer compare.

Q9. How do you get the session from a server-side redirect into an SPA safely?

Not by putting the JWT in the URL. We redirect with a random 256-bit ticket in the URL fragment (never sent to servers). Only its hash is stored, it lives 60 seconds, it's redeemed once via an atomic find-and-clear, and it's bound to a PKCE-style challenge whose verifier only the originating tab holds. The SPA scrubs the URL immediately and POSTs ticket + verifier for the real tokens.

Q10. What is an open redirect, and why does it matter in OAuth?

An endpoint that redirects anywhere a parameter says. In OAuth the redirect carries a credential (code or ticket), so an attacker-chosen target steals it. Providers require pre-registered exact redirect URIs. We apply the same rule: exact-match origin allowlist, and relative-only `next` paths. Pattern allowlists like `*.vercel.app` are unsafe because anyone can host there.

Q11. Where should an SPA store tokens?

The safest is not in JS at all: a backend-for-frontend keeps tokens server-side and gives the browser an httpOnly, Secure, SameSite cookie, with CSRF protection. Next best: in memory with short-lived tokens and silent refresh. `localStorage` is readable by any XSS. Our app used it until we moved the access token into memory (15 min) with silent refresh from an httpOnly cookie, the "next best" option (section 15). A same-site API domain or a BFF is the step after that.

Q12. SameSite Lax vs Strict vs None?

Strict: never sent on cross-site requests, even top-level navigations. Lax: sent on cross-site top-level GET navigations, not on cross-site fetch/POST/iframes. None: always (requires Secure), i.e. third-party. Our OAuth transaction cookie must be Lax because the provider's callback is a cross-site top-level GET.

Q13. What is the `alg:none` / algorithm-confusion attack?

Some libraries trusted the token header's `alg`. With `none` the token is unsigned; with HS256 an attacker uses the server's RSA *public* key as an HMAC secret to forge a token. Always pin allowed algorithms on verify. We pin RS256 and have tests for both attacks.

Q14. Confidential vs public client?

A confidential client can keep a secret (server-side): it authenticates to the token endpoint with a client secret. A public client (SPA, mobile app) can't, so it relies on PKCE alone. We are confidential, since the backend does the exchange, and we still use PKCE (defence in depth; OAuth 2.1 recommends it for everyone).

Q15. Access token vs refresh token vs ID token?

Access token: short-lived authorization for an API (audience = the API). Refresh token: long-lived credential to get new access tokens, sent only to the auth server, ideally rotated. ID token: proof of authentication for the client (audience = the client), not to be sent to APIs. Inside our app we issue our own access + refresh tokens after verifying the provider's ID token, and we don't keep the provider's tokens.

Q16. How does your backend authenticate your own Python agent service?

Per run, Express mints a 5-minute HS256 JWT with `aud: agent-tools`, explicit scopes and `act` = the user it acts for, signed with a dedicated secret. Internal routes use `requireAgentScope(...)` instead of user auth, so the agent never holds a user's token, and each route demands specific scopes (least privilege).

Q17. How do you access customers' AWS accounts without storing their keys?

`sts:AssumeRole` into a role the customer creates that trusts our account, with an **ExternalId** to prevent the confused-deputy problem (another customer tricking us into using your role). We get 1-hour temporary credentials. For CI, GitHub Actions uses OIDC federation: AWS trusts GitHub's token issuer, constrained by `aud` and `sub` (repo/branch), so no AWS keys live in GitHub.

Q18. Why hash password-reset tokens and login tickets in the database?

They are bearer secrets. If the DB leaks (backup, injection, insider), hashes are useless for login. Since the tokens are 256-bit random, a fast SHA-256 is enough (no bcrypt needed, because there's nothing to brute-force). Passwords are low-entropy, so they need a slow hash (bcrypt, cost 12).

Q19. A user signs in with Google but their email is unverified at the provider. What happens?

If we already know their `sub`, they sign in. Otherwise we refuse to link or create (403, "verify an email with the provider"). Trusting an unverified email would let anyone claim someone else's account.

Q20. How did you make concurrent first logins safe?

A unique index on `{provider, subject}` (and on user email). If two callbacks race, the loser gets MongoDB error 11000; we delete any half-created user and retry once, which then finds the winner's rows. The database enforces the invariant; the code just recovers.

Q21. How would you add "Sign in with Microsoft"?

Register an Entra ID app, add a provider object with `authorizeUrl` / `fetchProfile` (OIDC, so reuse the ID-token verifier with Microsoft's JWKS and issuer; for multi-tenant, validate the issuer per tenant), add `'microsoft'` to the `AuthIdentity` enum, and set env vars. The controller, linking, ticket and UI need no change.

Q22. Authentication vs authorization in your codebase?

AuthN: `requireAuth` establishes `req.user` from a verified JWT. AuthZ: `authorize(roles.X)` compares role rank per route (viewer < devops < architect < admin < owner < superadmin), plus workspace scoping in controllers and per-feature dashboard modules derived from role.

Q23. How did you test the security properties, not just the happy path?

Unit tests for each attack (wrong audience, wrong nonce, alg:none, HS256 confusion, foreign key, tampered cookie, wrong state, attacker origin), an RFC test vector for PKCE, an HTTP end-to-end reproduction of the pre-hijack attack against a real database, and a real-browser run through the whole redirect chain.

Q24. What happens when Google rotates its signing keys?

New tokens carry a new `kid`. Our cache misses on that kid and refetches the JWKS once, so verification continues without downtime. Otherwise keys are cached for Google's `Cache-Control: max-age`.

Q25. Why does login on Lambda need a cookie rather than an in-memory session?

Lambda containers are ephemeral, and two requests can hit different containers. The state/nonce/verifier must survive from `/start` to `/callback`, so we put them in a signed, httpOnly, 10-minute cookie: stateless on the server, tamper-proof, invisible to JavaScript. The alternative is a shared store such as DynamoDB or Redis.

Q26. Why does your callback redirect with an error *code* instead of the error message?

Anything in a URL can be forged by whoever sends the link. If the login page printed `?oauth_error=` verbatim, an attacker could make our own trusted page say "Your account is locked, call this number". That is content spoofing, and React's escaping doesn't prevent it, because it isn't script injection. So the server sends a code and the page maps it to fixed text. Unknown codes get a generic message.

Q27. What happens if Google is slow or down during sign-in?

Every outbound call has a 10-second `AbortSignal.timeout`. On timeout the callback logs an error-level line and redirects the user to the login page with `provider_error` ("try again in a moment"). Without the timeout a hung connection would hold the Lambda until its own limit: billed time, a stuck user, and concurrency used up during an outage.

Q28. What does the `__Host-` cookie prefix give you?

The browser only accepts a `__Host-` cookie if it is `Secure`, has `Path=/`, and has no `Domain` attribute, so it is locked to exactly one host. A compromised sibling subdomain can't set or overwrite it (cookie tossing), which would otherwise let an attacker plant a sign-in state in the victim's browser.

Q29. How do you avoid one misconfiguration taking down the whole API?

Fail closed, but narrowly. If the OAuth state secret is still the public dev default on a deployed server, only social sign-in switches off (no buttons; `/start` returns `not_enabled`) and an error is logged. Password login and every other route keep working. Throwing at startup would have been “secure” but turned a missing setting into a full outage.

14. Cheat sheet & glossary

14.1 Files changed or added

File	Role
`IAAS backend/src/services/oauthProviders.js` **NEW**	Google (OIDC) + GitHub provider definitions; Google ID-token verification with JWKS cache; timing-safe compare.
`IAAS backend/src/services/oauthLogin.js` **NEW**	PKCE, transaction cookie, return-path and origin allowlists, account resolution/linking, ticket issue/redeem.
`IAAS backend/src/controllers/oauthController.js` **NEW**	The four HTTP handlers.
`IAAS backend/src/models/AuthIdentity.js` **NEW**	External identities, unique `{provider, subject}`.
`IAAS backend/src/models/OAuthLoginTicket.js` **NEW**	One row per sign-in hand-off: ticket hash, client challenge, notice, TTL expiry.
`IAAS backend/src/models/User.js`	Optional password; `emailVerified`; `sessionVersion`; safe `comparePassword`.
`IAAS backend/src/utils/tokens.js`	`sv` claim; 401 instead of 500 on bad JWT; refresh-type check; `assertSessionCurrent`.
`middleware/auth.js`, `authController.js`, `githubController.js`	Session-version checks; reset revokes sessions; federated-only login message.
`config/env.js`, `.env.example`, `routes/authRoutes.js`	New config + routes; `IS_DEPLOYED`; social sign-in fails closed without a real state secret.
`src/auth/authClient.ts`, `src/auth/AuthPage.tsx`, `src/styles/auth-studio.css`	Buttons, SPA-side PKCE, ticket redemption, error display.
`IAAS backend/test/oauthLogin.test.js` **NEW**	22 security-focused unit tests.
`IAAS backend/test/oauthFlow.integration.test.js` **NEW**	6 HTTP + MongoDB tests (skip without `OAUTH_IT_MONGODB_URI`).
`IAAS backend/scripts/e2e/oauth-browser-check.mjs` **NEW**	Real-Chrome check, `npm run e2e:oauth-browser`.

14.2 Glossary

Authorization code	One-time, short-lived code returned via the browser, exchanged server-side for tokens.
PKCE	Proof Key for Code Exchange (RFC 7636): verifier/challenge pair that binds the code to whoever started the flow.
state	Anti-CSRF value tied to the user's browser, round-tripped through the provider.
nonce	Value placed into the ID token to bind it to one authentication request.
ID token	OIDC JWT proving authentication, audience = your client id.
JWKS	JSON Web Key Set, the provider's published public keys, selected by `kid`.
sub (subject)	Provider's stable, unique user identifier.
Scope	The permissions requested (`openid email profile`, `read:user`, `repo`...).
Confidential / public client	Can / cannot keep a client secret.

Redirect URI	Pre-registered exact URL the provider may send codes to.
Confused deputy	A trusted party tricked into using its authority for someone else (token from another app; AWS role without ExternalId).
Login CSRF	Forcing a victim to be logged into the attacker's account.
Pre-account hijacking	Attacker prepares an account with the victim's identifier before the victim registers.
Session version	Per-user counter embedded in JWTs; incrementing it revokes all earlier tokens.
Workload identity federation	A workload (CI job) exchanges its platform-issued OIDC token for cloud credentials; no stored keys.

14.3 Specs worth citing

RFC 6749 (OAuth 2.0) · RFC 7636 (PKCE) · RFC 9700 (OAuth 2.0 Security Best Current Practice, 2025) · OAuth 2.1 draft · OpenID Connect Core 1.0 · RFC 7519 (JWT) · RFC 7517 (JWK) · RFC 8725 (JWT Best Current Practices) · Sudhodanan & Paverd, "Pre-hijacked accounts", USENIX Security 2022 · GitHub changelog 2025-07-14 "PKCE support for OAuth and GitHub App authentication".

15. Token storage hardening: access token out of `localStorage` NEW

This section is a self-contained record of one change: the access JWT is **no longer saved in** `localStorage`, for email/password login and Google/GitHub login alike. It lives only in memory, and the httpOnly refresh cookie restores it after a reload. It was the first item in section 12.2.

> **Status (17 Oct 2026)**
> Implemented and tested locally (unit, integration and real-browser tests all pass). **Not yet deployed to production**. Read 15.7 first: one production issue (Safari) should be decided before release.

15.1 Before and after

Aspect	Before	After
Where the access token lives	`localStorage['infra-auth-token']`, readable by any script on the page	A module variable in `src/auth/authClient.ts`; gone when the tab closes. Old copies are deleted on load.
Access-token lifetime	6 h	15 min default (`JWT_ACCESS_EXPIRES_IN`)
Reload / new tab	Read the token from storage, check it with `/auth/me`	`POST /auth/refresh` with the httpOnly cookie → new access token + user
Token expires mid-session	Requests failed with 401 until the next reload	`authFetch` refreshes once and retries the request; parallel requests share one refresh
Refresh cookie	`HttpOnly; SameSite=Lax`, never actually used by the SPA	`HttpOnly; SameSite=None; Secure` when deployed (cross-site), `HttpOnly; SameSite=Lax` locally
Logout	`res.clearCookie('refreshToken')`	Cleared with the same SameSite/Secure attributes. Otherwise a cross-site response can't overwrite a `SameSite=None` cookie and logout would silently leave it.
CORS	Exact list **plus** any `*.cloudfront.net`, `*.vercel.app`, `*.execute-api.*`, all with credentials	**Exact, listed origins only**; on a deployed API also no http/loopback origins
GitHub-connect link	Access token in `?token=`; also printed with `console.log`	Still in `?token=` (a navigation can't send headers), but if it has expired the backend falls back to the refresh cookie. `console.log` removed.

15.2 How it works now

1. **Sign in** (password, register, or the OAuth ticket exchange): the response carries `{accessToken, user}`; the refresh token only ever travels in the `Set-Cookie` header. `storeAuthSession()` keeps the token in memory and caches the *user* in `localStorage`. The user object is not a credential; it only lets the UI render roles before the API answers.
2. **Every API call** goes through `authFetch()`, which adds `Authorization: Bearer`. On a 401 it calls `refreshSession()` once and retries. If the refresh fails, the session is cleared and the original 401 goes back to the caller.
3. **Reload or new tab**: memory is empty. `validateStoredSession()` (the route guard in `App.tsx`) calls `refreshSession()` when a cached user exists. With no cached user, the visitor has never signed in here, so it answers "signed out" without a request.

4. **Logout**: `POST /auth/logout` clears the cookie (same attributes as when it was set); memory and the cached user are cleared.

```
src/auth/authClient.ts
let accessToken: string | null = null;            // never written to storage

export async function authFetch(url, init = {}) {
  const send = () => fetch(url, { ...init, credentials: 'include',
    headers: { ...(accessToken ? { Authorization: `Bearer ${accessToken}` } : {}), ...init.headers } });
  const response = await send();
  if (response.status !== 401) return response;
  return (await refreshSession()) ? send() : response;   // one refresh, one retry
}
// refreshSession(): POST /auth/refresh (cookie) → storeAuthSession(); concurrent callers share one in-flight
promise
```

15.3 Refresh cookie and CORS: why they changed together

In production the SPA (`d3pgg5abvvdatt.cloudfront.net` / `codemanus.in`) and the API (`…execute-api.ap-south-1.amazonaws.com`) are **different sites**. A `SameSite=Lax` cookie is never sent on a cross-site `fetch`, so `/auth/refresh` could not work. The cookie therefore became `SameSite=None`, which browsers only accept with `Secure`.

Setting	Local dev	Deployed (`IS_DEPLOYED`)	Why
Refresh cookie `SameSite`	`Lax`	`None`	Locally page and API are both on `127.0.0.1` (same site). Deployed they are cross-site.
Refresh cookie `Secure`	no (plain http)	yes	Required for `SameSite=None`.
CORS with credentials	Listed origins incl. `http://localhost:5173`, `http://127.0.0.1:5173`	Listed **https, non-loopback** origins only	See below.

> **The attack this prevents**
> With a `SameSite=None` cookie, the cookie is sent to the API from *any* site. Whether that site can **read** the response is decided by CORS. The old CORS rule allowed every `*.cloudfront.net` and `*.vercel.app` origin with credentials. Anyone can deploy there, so such a page could call `POST /auth/refresh` as the visitor and read back a valid access token: full account takeover. CORS is now exact-match only (`IAAS backend/src/utils/cors.js`). On a deployed API, `http://localhost` origins are dropped too, so nothing that happens to run on a visitor's machine can read their session. Unit-tested in `test/cors.test.js`.

15.4 Local development

- **Open** `http://127.0.0.1:5173`. The dev API is `127.0.0.1:4001`, and to the browser `localhost` is a different site, so a page on `localhost` would never send the refresh cookie and every reload would sign you out. `src/main.tsx` therefore redirects a dev page on `localhost` to `127.0.0.1` (dev builds only).
- Local OAuth callbacks use the same host: `http://127.0.0.1:4001/api/v1/auth/oauth/{google,github}/callback`.
- The local `.env` can set a longer `JWT_ACCESS_EXPIRES_IN`; the refresh logic works with any value.
- Restart Vite after editing `authClient.ts`: hot reload can leave two copies of the module, each with its own in-memory token.

15.5 Files changed

File	Change
`src/auth/authClient.ts`	In-memory token; `getAccessToken`, `refreshSession` (deduplicated), `authFetch` (refresh + retry once); `validateStoredSession` restores from the cookie; legacy `infra-auth-token` removed on load.
`src/utils/apiClient.ts`, `src/dashboard/ticketApi.ts`, `src/utils/terraformPayloadApi.ts`	Use `authFetch` instead of reading the token themselves.
`src/github/githubApi.ts`	`getAccessToken()`; removed the `console.log` that printed the token.
`src/main.tsx`	Dev-only `localhost` → `127.0.0.1` redirect.
`IAAS backend/src/utils/tokens.js`	Shared refresh-cookie options (None+Secure deployed, Lax locally); `clearRefreshCookie()`.
`IAAS backend/src/controllers/authController.js`	Logout and password reset clear the cookie with matching attributes.
`IAAS backend/src/utils/cors.js`	Exact-match origins only; deployed API drops http/loopback.
`IAAS backend/src/controllers/githubController.js`	GitHub-connect falls back to the refresh cookie when the URL's access token has expired.
`IAAS backend/src/config/env.js`, `.env.example`	Default `JWT_ACCESS_EXPIRES_IN` 6h → 15m.
`IAAS backend/test/cors.test.js` NEW, `scripts/e2e/oauth-browser-check.mjs`	CORS tests; 8 new browser checks (below).

15.6 How it was verified

- **Backend:** 78/78 with MongoDB (72 + 1 skipped without). **Frontend:** type-check, production build and 57/57 unit tests.
- **HTTP, against the local server:** register returns the access token in the body but never the refresh token; cookie is `HttpOnly; SameSite=Lax`; `/auth/refresh` works with only the cookie; an unlisted origin gets no `Access-Control-Allow-Origin`; after logout, refresh returns 401.
- **Deployed mode (`NODE_ENV=production`):** cookie set and cleared with `SameSite=None; Secure`; CORS allows the CloudFront origin and refuses `http://localhost:5173` and `https://x.cloudfront.net`.
- **Real Chrome (21/21):** no JWT in `localStorage`/`sessionStorage` after Google or password login; reload stays signed in; an expired token is refreshed and the request retried; a reload after logout lands on `/login`.
- **Not tested:** against the live production site, or in Safari.

15.7 Production notes: read before deploying

1. Safari users would be signed out on every reload

In production the API is a different site from the SPA, so the refresh cookie is a *third-party* cookie. Safari blocks all third-party cookies by default; so do Brave, and Chrome or Firefox when the user turns that on. In those browsers `/auth/refresh` gets no cookie, and the user must sign in again after every reload or new tab. Chrome, Edge and Firefox with default settings work. **Proper fix:** serve the API from the SPA's own site, either with a CloudFront behaviour that routes `/api/*` to API Gateway, or `api.codemanus.in` with the SPA on `codemanus.in`. The cookie is then first-party and can go back to `SameSite=Lax`. Recommended before release.

#	Note	What to do
2	Existing users sign in once after the deploy.	Their old `localStorage` token is deleted, and their old `Lax` cookie was never usable cross-site. Expected; no action needed.
3	Wildcard CORS origins are gone.	Add every frontend users or testers open (e.g. Vercel previews, `https://codemanus.in`) to `CLIENT_ORIGINS`, or it can't call the API.
4	The 15 min default applies only if the Lambda doesn't set `JWT_ACCESS_EXPIRES_IN`.	Check the Lambda environment; remove or lower any 6h/1d value there.
5	Deploy both halves together.	The new SPA needs the new cookie settings, and the stricter CORS needs `CLIENT_ORIGINS` to be correct. Deploy backend and frontend in the same release.
6	Smoke test after deploy.	Sign in (password and GitHub), reload, open a second tab, leave a tab idle for over 15 min and use it, log out and reload. Then repeat in Safari to confirm note 1.

15.8 Interview angle

Q. Why not just keep the token in `localStorage` with a short expiry?

Expiry limits how long a stolen token works, not whether it can be stolen. Any XSS can read `localStorage` and send the token away. In memory, a script can still make requests while the page is open, but it can't lift a credential that outlives the tab. The long-lived refresh token is in an httpOnly cookie that JavaScript can't read at all.

Q. Your refresh cookie is `SameSite=None`. Isn't that a CSRF risk?

The cookie is sent cross-site, so a hostile page can *trigger* `/auth/refresh`. But the response (the access token) is only readable by origins CORS allows, and CORS is now an exact list, without the old `*.cloudfront.net` / `*.vercel.app` wildcards that would have made it a takeover. Authenticated API routes need the Bearer token, which a cross-site page doesn't have. A same-site API domain with `Lax` would be better still.

Q. How do you stop ten parallel requests from all refreshing at once when the token expires?

`refreshSession()` keeps the in-flight promise; every 401 awaits the same one, and each request retries once with the new token. A failed refresh clears the session instead of looping.